



Trade Records of Cyrene

In the prosperous ancient city of Cyrene, merchants continuously record the prices of goods arriving at the marketplace. However, instead of writing them down neatly, they overwrite old records on clay tablets.

Each time a new price arrives, a merchant may write it on any tablet, replacing whatever was written before. The goal is to end up with a well-organized record of prices despite this chaotic process.

Problem Description

You are given an array of length N , representing clay tablets in the marketplace. Initially, all tablets are empty.

A sequence of values (representing prices) between 0 and $M - 1$ arrives one by one. For each incoming value, you must choose a position in the array and write it there, overwriting the previous value.

Your goal is to ensure that:

- Every position in the array is written to at least once.
- After some number of operations, the array becomes sorted: $A_0 \leq A_1 \leq \dots \leq A_{N-1}$
- You must achieve this using **at most $N \times M$ operations**.

Interaction

You need to implement two procedures:

`init(N, M)`

- N : number of tablets
- M : number of possible values (prices range from 0 to $M - 1$)

This is called once at the beginning.

`place(value)`

- `value`: the incoming price

This function is called repeatedly. For each call, you must return the index where you want to write this value.

- The function is called at most $N \times M$ times.
- Once the array becomes sorted and fully filled, the process stops.

The system may behave adaptively depending on your previous choices.

Input (Sample Grader)

- First line: $N \ M$
- Second line: sequence $v[0], v[1], \dots, v[N \times M - 1]$ representing incoming values

Output (Sample Grader)

- Final array $A[0], A[1], \dots, A[N-1]$

Constraints

- $1 \leq N, M \leq 5000$

Subtasks

Test group	Points	Constraints
1	7	$N = 2$
2	14	$M = 2$
3	19	$M = 3$
4	24	$N = 5000$, The grader is not adaptive
5	36	No additional constraints

Examples

Example 1

`init(3, 5)` Initially: $A = [-1, -1, -1]$

Operations: `place(3) → 1`

$A = [-1, 3, -1]$

`place(1) → 0`

$A = [1, 3, -1]$

`place(2) → 1`

$A = [1, 2, -1]$

`place(4) → 2`

$A = [1, 2, 4]$

The array is now sorted and fully filled.

Total operations used: $4 \leq 3 \times 5 = 15$.

Notes

- You are free to overwrite any position multiple times.
- The challenge is to handle the sequence strategically so that you end up with a sorted array within the allowed number of operations.